

✓ movie recommendation

OBJECTIVE

The objective of a movie recommendation system in machine learning is to personalize movie suggestions, enhance user experience, and increase engagement by analyzing user preferences. It aims to help users discover new content, improve retention through relevant recommendations, and leverage collaborative filtering to reflect similar users tastes.

Recommendation system is a system that seeks to predict or filter the preference according to the user's choice. Recommendation system are utilized in a variety including movies, music, books, research, articles , search queries, social tags in general.

Recommendation system produces a list of recommendations in two ways,

collaboration filtering is a filtering approaches builds a model from the user's past behavior as well as similar decision made by other user. this model os used to predict items that user may have an intrest in .

content-based filtering approaches uses a series of discrete characteristics of an item in order to recomend assitional item with similar properties . The filtering methods are tottaly based on a description of the item and a profile of the user's preferences

Double-click (or enter) to edit

✓ Import Library

```
import pandas as pd
```

```
import numpy as np
```

✓ Import Data

```
data = pd.read_csv("Movies Recommendation.csv")
```

✓ Describe Data

```
data.head()
```



	Movie_ID	Movie_Title	Movie_Genre	Movie_Language	Movie_Budget	Movie_Popularity
0	1	Four Rooms	Crime Comedy	en	4000000	22.876230
1	2	Star Wars	Adventure Action Science Fiction	en	11000000	126.393695
2	3	Finding Nemo	Animation Family	en	94000000	85.688789
3	4	Forrest Gump	Comedy Drama Romance	en	55000000	138.133331
4	5	American Beauty	Drama	en	15000000	80.878605

5 rows × 21 columns

```
data.info()
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4760 entries, 0 to 4759
Data columns (total 21 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Movie_ID                             4760 non-null   int64
1   Movie_Title                           4760 non-null   object
2   Movie_Genre                           4760 non-null   object
3   Movie_Language                         4760 non-null   object
4   Movie_Budget                           4760 non-null   int64
5   Movie_Popularity                       4760 non-null   float64
6   Movie_Release_Date                     4760 non-null   object
7   Movie_Revenue                           4760 non-null   int64
8   Movie_Runtime                           4758 non-null   float64
9   Movie_Vote                             4760 non-null   float64
10  Movie_Vote_Count                       4760 non-null   int64
```

```

11  Movie_Homepage          1699 non-null  object
12  Movie_Keywords         4373 non-null  object
13  Movie_Overview         4757 non-null  object
14  Movie_Production_House  4760 non-null  object
15  Movie_Production_Country 4760 non-null  object
16  Movie_Spoken_Language  4760 non-null  object
17  Movie_Tagline          3942 non-null  object
18  Movie_Cast             4733 non-null  object
19  Movie_Crew             4760 non-null  object
20  Movie_Director         4738 non-null  object
dtypes: float64(3), int64(4), object(14)
memory usage: 781.1+ KB

```

```
data.shape
```

```
↪ (4760, 21)
```

```
data.columns
```

```

↪ Index(['Movie_ID', 'Movie_Title', 'Movie_Genre', 'Movie_Language',
        'Movie_Budget', 'Movie_Popularity', 'Movie_Release_Date',
        'Movie_Revenue', 'Movie_Runtime', 'Movie_Vote', 'Movie_Vote_Count',
        'Movie_Homepage', 'Movie_Keywords', 'Movie_Overview',
        'Movie_Production_House', 'Movie_Production_Country',
        'Movie_Spoken_Language', 'Movie_Tagline', 'Movie_Cast', 'Movie_Crew',
        'Movie_Director'],
        dtype='object')

```

✓ Data Visualization

Get Feature Selection

```
data_feature = data[['Movie_Genre', 'Movie_Keywords', 'Movie_Tagline', 'Movie_Cast', 'Movie_D
```

```
data_feature.shape
```

```
↪ (4760, 5)
```

```
data_feature
```



	Movie_Genre	Movie_Keywords	Movie_Tagline	Movie_Cast	Movie_Director
0	Crime Comedy	hotel new year's eve witch bet hotel room	Twelve outrageous guests. Four scandalous requ...	Tim Roth Antonio Banderas Jennifer Beals Madon...	Allison Anders
1	Adventure Action Science Fiction	android galaxy hermit death star lightsaber	A long time ago in a galaxy far, far away...	Mark Hamill Harrison Ford Carrie Fisher Peter ...	George Lucas
2	Animation Family	father son relationship harbor underwater fish...	There are 3.7 trillion fish in the ocean, they...	Albert Brooks Ellen DeGeneres Alexander Gould ...	Andrew Stanton
3	Comedy Drama Romance	vietnam veteran hippie mentally disabled runni...	The world will never be the same, once you've ...	Tom Hanks Robin Wright Gary Sinise Mykelti Wil...	Robert Zemeckis
4	Drama	male nudity female nudity adultery midlife cri...	Look closer.	Kevin Spacey Annette Bening Thora Birch Wes Be...	Sam Mendes
...	...	...	...	...	...
4755	Horror		The hot spot where Satan's waitin'.	Lisa Hart Carroll Michael Des Barres Paul Drak...	Pece Dingo
4756	Comedy Family Drama		It's better to stand out than to fit in.	Roni Akurati Brighton Sharbino Jason Lee Anjul...	Frank Lotito
4757	Thriller Drama	christian film sex trafficking	She never knew it could happen to her...	Nicole Smolen Kim Baldwin Ariana Stephens Brys...	Jaco Booyens
4758	Family				

Next
steps:

[Generate code with data_feature](#)

[View recommended plots](#)

[New interactive sheet](#)



✓ Data preprocessing

```
x = data_feature['Movie_Genre'] + ' ' + data_feature['Movie_Keywords'] + ' ' + data_feat
```

```
print(x)
```

```
➡ 0      Crime Comedy hotel new year's eve witch bet ho...
   1      Adventure Action Science Fiction android galax...
   2      Animation Family father son relationship harbo...
   3      Comedy Drama Romance vietnam veteran hippie me...
   4      Drama male nudity female nudity adultery midli...
      ...
  4755    Horror The hot spot where Satan's waitin'. Li...
  4756    Comedy Family Drama It's better to stand out ...
  4757    Thriller Drama christian film sex trafficking ...
  4758                                Family
  4759    Documentary music actors legendary performer cl...
Length: 4760, dtype: object
```

```
x.shape
```

```
➡ (4760,)
```

✓ Define Target Variable (y) and Feature Variables (X)

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
Tf = TfidfVectorizer()
```

```
x = Tf.fit_transform(x)
```

```
x.shape
```

```
x.shape
```

```
➡ (4760, 17258)
```

```
print(x)
```

```
➡ (0, 3583)    0.06486754376295062
   (0, 3240)    0.04527089872278055
   (0, 7213)    0.25146675849405775
   (0, 10898)   0.17625708810661284
   (0, 17052)   0.26079573581490934
   (0, 5059)    0.29553419178998613
   (0, 16862)   0.12768803549311025
```

```

(0, 1595)      0.15687561633854538
(0, 13052)     0.1465525095337543
(0, 15708)     0.17654247479915475
(0, 11362)     0.18801785343006192
(0, 6463)      0.18801785343006192
(0, 5662)      0.1465525095337543
(0, 13467)     0.19712637387361423
(0, 12731)     0.19712637387361423
(0, 614)       0.07642616241686973
(0, 11244)     0.08262965296941757
(0, 9206)      0.15186283580984414
(0, 1495)      0.19712637387361423
(0, 7454)      0.14745635785412262
(0, 7071)      0.19822417598406614
(0, 5499)      0.11454057510303811
(0, 3878)      0.11998399582562203
(0, 11242)     0.07277788238484746
(0, 15219)     0.09800472886453934
:              :
(4757, 3485)   0.199161573117024
(4757, 1184)   0.18890726729447022
(4757, 14568)  0.24255077606762876
(4757, 15508)  0.24255077606762876
(4757, 5802)   0.24255077606762876
(4757, 819)    0.27474840155297187
(4757, 14195)  0.28805858134028367
(4757, 2227)   0.28805858134028367
(4757, 7691)   0.28805858134028367
(4757, 1932)   0.28805858134028367
(4758, 5238)   1.0
(4759, 10666)  0.15888268987343043
(4759, 1490)   0.21197258705292082
(4759, 15431)  0.19628653185946862
(4759, 5690)   0.19534291014627303
(4759, 14051)  0.20084315377640435
(4759, 4358)   0.18306542312175342
(4759, 10761)  0.3126617295732147
(4759, 7130)   0.26419662449963793
(4759, 3058)   0.2812896191863103
(4759, 14062)  0.3237911628497312
(4759, 8902)   0.3040290704566037
(4759, 205)    0.3237911628497312
(4759, 11708)  0.33947721804318337
(4759, 11264)  0.33947721804318337

```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
sim = cosine_similarity(x)
```

```
sim
```

```

➡ array([[1.          , 0.01351235, 0.03570468, ..., 0.          , 0.          ,
          0.          ],
          [0.01351235, 1.          , 0.00806674, ..., 0.          , 0.          ,
          0.          ],
          [0.03570468, 0.00806674, 1.          , ..., 0.          , 0.08014876,
          0.          ],

```

```
...,
[0., 0., 0., ..., 1., 0.,
 0. ],
[0., 0., 0.08014876, ..., 0., 1.,
 0. ],
[0., 0., 0., ..., 0., 0.,
 1. ]])
```

```
sim.shape
```

 (4760, 4760)

✓ Train Test Split

```
movie_name = input("Enter your favourite movie name: ")
```

```
➡ Enter your favourite movie name: jungle
```

```
Movie_list = data['Movie_Title'].tolist()
```

- ✓ modeling

```
import difflib
```

```
Movie_recomended = difflib.get_close_matches(movie_name,Movie_list)
```

Movie_recomended

➡ ['Tangled', 'Gunless', 'Dune']

```
matching_movie = Movie_recomended[0]
```

matching_movie

 'Tangled'

```
index_of_movie = data[data.Movie_Title == matching_movie]['Movie_ID'].values[0]
```

index_of_movie

 3209

✓ Model Evaluation

```
#Getting list of similar movies  
score = list(enumerate(sim[index_of_movie]))
```

score



```
(946, 0.01281947955709329),  
(947, 0.0),  
(948, 0.005110179169742071),  
(949, 0.0),  
(950, 0.0),  
(951, 0.0034261588594436576),  
(952, 0.0),  
(953, 0.01917722555359123),  
(954, 0.0),  
(955, 0.004785308251302480)
```

```
len(score)
```

```
↩ 4760
```

```
similar_movies = sorted(score, key = lambda x:x[1], reverse = True)
```

```
similar_movies
```

```
↩
```

```
(171, 0.014233116040642469),
(2493, 0.0142176165183974),
(819, 0.01421190064861178),
(3023, 0.014186399759535789),
(2410, 0.01413221909373693),
(956, 0.014114085615668815),
(4097, 0.014107399856208753),
(154, 0.014074540776756331),
(2552, 0.014018720177303184),
(1206, 0.014016885409272125),
(4207, 0.0139806676665184),
(2862, 0.013973899431841541),
(4086, 0.01394002196954933),
(1641, 0.013904525844238185),
(3120, 0.013882092021821621),
(1518, 0.013874205830678069),
(3114, 0.01386156315679832),
(35, 0.01383564219127918),
...]
```

✓ prediction

```
print("40 movies suggested for you")
i = 1
for movie in similar_movies:
    index = movie[0]
    title_from_index = data[data.index == index]['Movie_Title'].values[0]
    if(i<41):
        print(i, '.', title_from_index)
        i+=1
```

```
⇒ 40 movies suggested for you
1 . Dinner for Schmucks
2 . The Campaign
3 . Gentlemen Broncos
4 . How Do You Know
5 . R.I.P.D.
6 . The 40 Year Old Virgin
7 . The BFG
8 . Youth in Revolt
9 . The Good Heart
10 . Anchorman: The Legend of Ron Burgundy
11 . Anchorman 2: The Legend Continues
12 . Date Night
13 . Alexander and the Terrible, Horrible, No Good, Very Bad Day
14 . The Incredible Burt Wonderstone
15 . The Oh in Ohio
16 . The Tale of Despereaux
17 . Trainwreck
18 . The Number 23
19 . Austin Powers: The Spy Who Shagged Me
20 . Varsity Blues
21 . The DUFF
22 . Super Troopers
23 . The Hangover Part II
24 . Escape from Alcatraz
```

```
25 . Mystery, Alaska
26 . Due Date
27 . The Muppets
28 . Beerfest
29 . The Hangover
30 . Knocked Up
31 . Wanderlust
32 . Tim and Eric's Billion Dollar Movie
33 . Our Idiot Brother
34 . Pearl Harbor
35 . Gamer
36 . Birdman
37 . Puss in Boots
38 . Wish I Was Here
39 . Meet the Fockers
40 . Meet the Parents
```

```
movie_name = input("Enter your favourite movie name: ")
titles = data['Movie_Title'].values
matching_movie = Movie_recomended[0]
index_of_movie = data[data.Movie_Title == matching_movie]['Movie_ID'].values[0]
score = list(enumerate(sim[index_of_movie]))
similar_movies = sorted(score, key = lambda x:x[1], reverse = True)
print("15 movies suggested for you")
i = 1
for movie in similar_movies:
    index = movie[0]
    title_from_index = data[data.index == index]['Movie_Title'].values[0]
    if(i<16):
        print(i, '.', title_from_index)
        i+=1
```



```
Enter your favourite movie name: jungle
15 movies suggested for you
1 . Dinner for Schmucks
2 . The Campaign
3 . Gentlemen Broncos
4 . How Do You Know
5 . R.I.P.D.
6 . The 40 Year Old Virgin
7 . The BFG
8 . Youth in Revolt
9 . The Good Heart
10 . Anchorman: The Legend of Ron Burgundy
11 . Anchorman 2: The Legend Continues
12 . Date Night
13 . Alexander and the Terrible, Horrible, No Good, Very Bad Day
14 . The Incredible Burt Wonderstone
15 . The Oh in Ohio
```

✓ Explanation

Start coding or [generate](#) with AI.

